



Codex

Python'un İe Aktarma Gizemlerini Anlamak

Özet: Simya deneyleri yoluyla Python ie aktarımlarının kadim sanatını keşfedin. Dört kutsal gizemi ustalaşın: paket başlatma, ie aktarım yolları, mutlak ve göreceli erişim ve döngüsel bağımlılıkları kırma.

Sürüm: 2.0

İçindekiler

I	AI Talimatları	2
II	Giriş	4
III	Genel Talimatlar	5
IV	Zorunlu bölüm	7
IV.1	Bölüm I: Alembik	7
IV.2	Bölüm II: Damıtma.....	10
IV.3	Bölüm III: Büyük Dönüşüm	12
IV.4	Bölüm IV: Patlamayı Önlemek	14
V	Gönderim Talimatları	16

Bölüm I

AI Talimatları

● Bağlam

Öğrenme yolculuğunuz boyunca, AI birçok farklı görevde size yardımcı olabilir. Zaman ayırarak AI araçlarının çeşitli yeteneklerini ve çalışmalarınızı nasıl destekleyebileceklerini keşfedin. Ancak, bunlara her zaman dikkatli yaklaşın ve sonuçları eleştirel bir gözle değerlendirin. İster kod, ister dokümantasyon, fikirler veya teknik açıklamalar olsun, sorunuzun doğru bir şekilde formüle edildiğinden veya üretilen içeriğin doğru olduğundan asla tam olarak emin olamazsınız. Meslektaşlarınız, hatalardan ve kör noktalardan kaçınmanıza yardımcı olacak değerli bir kaynaktır.

● Ana mesaj

- ✎ Tekrarlayan veya sıkıcı görevleri azaltmak için yapay zekayı kullanın.
- ✎ Gelecekteki kariyerinize fayda sağlayacak, hem kodlama hem de kodlama dışı alanlarda komut verme becerilerinizi geliştirin.
- ✎ Yaygın riskleri, önyargıları ve etik sorunları daha iyi öngörebilmek ve önleyebilmek için yapay zeka sistemlerinin nasıl çalıştığını öğrenin.
- ✎ Akranlarınızla birlikte çalışarak hem teknik hem de sosyal becerilerinizi geliştirmeye devam edin.
- ✎ Yalnızca tam olarak anladığınız ve sorumluluğunu üstlenebileceğiniz yapay zeka tarafından üretilen içerikleri kullanın.

● Öğrenci kuralları:

- AI araçlarını keşfetmek ve nasıl çalıştıklarını anlamak için zaman ayırmalısınız; böylece bunları etik bir şekilde kullanabilir ve olası önyargıları azaltabilirsiniz.
- Komut vermeden önce sorunuz üzerinde düşünmelisiniz; bu, doğru kelimeleri kullanarak daha net, daha ayrıntılı ve daha alakalı komutlar yazmanıza yardımcı olur.
- AI tarafından üretilen her şeyi sistematik olarak kontrol etme, gözden geçirme, sorgulama ve test etme alışkanlığı geliştirmelisiniz.
- Her zaman akran değerlendirmesi almalısınız — yalnızca kendi doğrulamanıza güvenmeyin.

● Ařama ıktıları:

- Hem genel amaçlı hem de alana özgü komut satırı becerilerini geliştirin.
- AI araçlarını etkili bir şekilde kullanarak üretkenliğinizi artırın.
- Hesaplamalı düşünme, problem çözme, uyum sağlama ve işbirliği becerilerinizi güçlendirmeye devam edin.

● Yorumlar ve örnekler:

- Sınavlar, değerlendirmeler ve benzeri durumlarda, gerçek anlamda anladığınızı kanıtlamanız gereken durumlarla düzenli olarak karşılaşacaksınız. Hazırlıklı olun, hem teknik hem de kişilerarası becerilerinizi geliştirmeye devam edin.
- Gerekçelerinizi açıklamak ve akranlarınızla tartışmak, genellikle anlayışınızdaki eksiklikleri ortaya çıkarır. Akran öğrenimini bir öncelik haline getirin.
- AI araçları genellikle sizin özel bağlamınızı göz önünde bulundurmaz ve genel yanıtlar verme eğilimindedir. Sizinle aynı ortamı paylaşan akranlarınız, daha alakalı ve doğru içgörüler sunabilir.
- AI genellikle en olası cevabı üretmeye eğilimliyken, akranlarınız alternatif bakış açıları ve değerli nüanslar sunabilir. Onları bir kalite kontrol noktası olarak değerlendirin.

✓ İyi uygulama:

AI'ya řunu sorarım: “Bir sıralama işlevini nasıl test edebilirim?” Bana birkaç fikir verir. Bunları denerim ve sonuçları bir akranımla gözden geçiririm. Yaklaşımı birlikte iyileştiririz.

✗ Kötü uygulama:

AI'dan fonksiyonun tamamını yazmasını isterim, bunu projemde kopyalayıp yapıştırırım. Meslektaş değerlendirmesi sırasında, fonksiyonun ne yaptığını veya nedenini açıklayamam. Güvenilirliğimi yitiririm — ve projemden kalırım.

✓ Doğru uygulama:

AI'yı bir ayrıştırıcı tasarlamaya yardımcı olması için kullanırım. Ardından bir akranımla birlikte mantığını gözden geçiririz. İki hata buluruz ve birlikte yeniden yazarız — daha iyi, daha temiz ve tam olarak anlaşılır hale getiririz.

✗ Kötü uygulama:

Copilot'un projemin önemli bir kısmı için kodumu oluşturmalarına izin veriyorum. Kod derleniyor, ancak boruları nasıl işlediğini açıklayamıyorum. Değerlendirme sırasında gerekçelendiremediğim için projemden kalıyorum.

Bölüm II

Giriş

Alkimist Laboratuvarı'na hoş geldin, genç çırak!

Orta Çağ'da simyacılar, adi metalleri altına dönüştürmeye çalışırlardı. Bugün, bir Python simyacısı olarak, dağınık kodları düzenli, yeniden kullanılabilir sihirli formüllere dönüştürmeyi öğreneceksin. Tıpkı eski simyacıların grimoire'lerini (büyü kitaplarını) bölümlere ve alt bölümlere ayırmaları gibi, Python kodu da modüllere ve paketlere ayrılmalıdır.

Ama dikkat et! Simyacının yolu gizemlerle doludur:

- Basit bir klasörü sihirli bir pakete dönüştüren geçit olan `__init__.py`'yi kullanma
- Uzak bir modülden uzak bir modülün kodunu çağırma
- Mutlak ve göreceli içe aktarmaları yönetmek
- Döngüsel bağımlılıklardan kaçınmak

Bu kodeks'te, kendi simya laboratuvarınızı kuracaksınız: bu dört kutsal gizemi gösteren eksiksiz bir Python paketi.



İçe aktarmaların sadece çalışmasını sağlamakla kalmayıp, **nasıl çalıştığını** anlamaya odaklanın. Gerçek bir simyacı, dönüşüm sürecinin her adımını anlar!

Bölüm III

Genel Talimatlar

- Projeniz **Python 3.10 veya daha yeni** bir sürümde yazılmalıdır.
- Projeniz **flake8** kodlama standardına uymalıdır.
- Tüm kodlar kapsamlı tür açıklamaları içermelidir. Bunu mypy ile kontrol edin.
- Tüm standart sınıflar ve koleksiyonlar, yöntemleriyle birlikte (int, str, list, dict vb.) kullanılabilir.
- Tüm yerleşik işlevler izinlidir, ancak eval() ve exec() hariçtir.
- Yalnızca bu projede oluşturacağınız modüllerin ve dosyaların içe aktarılmasına izin verilir.
- sys.path'i değiştirmek yasaktır.

Bu proje, benzersiz bir dosya ve klasör yapısı oluşturacak dört bölüme ayrılmıştır. Projenin sonunda, ağacınız şöyle görünmelidir:

```
.
|-- alchemy
|   |-- _init_.py
|   |-- elements.py
|   |-- grimoire
|       |-- _init_.py
|       |-- dark_spellbook.py
|       |-- dark_validator.py
|       |-- light_spellbook.py
|       '-- light_validator.py
|-- potions.py
|-- 'transmutation
|   |-- _init_.py
|   '-- recipes.py
|-- elements.py
|-- ft_alembic_0.py
|-- ft_alembic_1.py
|-- ft_alembic_2.py
|-- ft_alembic_3.py
|-- ft_alembic_4.py
|-- ft_alembic_5.py
|-- ft_damıtma_0.py
|-- ft_distillation_1.py
|-- ft_kaboom_0.py
|-- ft_kaboom_1.py
|-- ft_transmutation_0.py
|-- ft_transmutation_1.py '-
- ft_transmutation_2.py
```



Tüm fonksiyonlar basit olmalı ve string döndürmelidir. Karmaşık mantığa değil, import kavramlarına odaklanın.

Bölüm IV Zorunlu

kısım

IV.1 Bölüm I: Alembic

Her simyacı, Alembic'inde basit malzemelerle işe başlar! Ancak bazen bazı malzemeler elinizin altında olurken, diğerleri olmayabilir. Bu ilk bölüm, yerel dosyalara ve modüllere erişmek için içe aktarma mekanizmalarını kullanmayı ele almaktadır.

İlk olarak, laboratuvar mimarimizin bileşenlerini (ve ardından test paketini) oluşturacağız.

elements.py dosyası şunları içerecektir:

- Sadece “Ateş elementi oluşturuldu” dizesini döndüren `create_fire()` işlevi.
- Sadece “Water element created” dizesini döndüren `create_water()` işlevi.

alchemy/elements.py dosyası şunları içerecektir:

- Sadece “Earth element created” dizesini döndüren `create_earth()` işlevi.
- Sadece “Air element created” dizesini döndüren `create_air()` işlevi.

alchemy/____init____.py dosyası şunları içerecektir:

- alchemy modülünü kısmen içe aktarmak için gerekli her şey. Bkz. `ft_alembic_4.py` .

Laboratuvar mimarimizi dikkatli bir şekilde test etmek için aşağıdaki altı komut dosyasını oluşturun:

- `ft_alembic_0.py`, `import ...` yapısını kullanarak `elements.py` dosyasına doğrudan erişecek ve ardından `fire` nesnesini oluşturacaktır.
- `ft_alembic_1.py` dosyası, `elements.py` dosyasına doğrudan erişmek için `from ... import ...` yapısını kullanacak ve ardından `water`'ı oluşturacaktır.
- `ft_alembic_2.py` dosyası, `alchemy/elements.py` dosyasına doğrudan erişmek için `import ...` yapısını kullanacak ve ardından `earth` nesnesini oluşturacaktır.

- ft_alembic_3.py dosyası, alchemy/elements.py dosyasına doğrudan erişmek için from ... import ... yapısını kullanacak ve ardından air nesnesini oluşturacaktır.
- ft_alembic_4.py dosyası, alchemy modülüne erişmek için import alchemy ifadesini kullanacak ve ardından air nesnesini oluşturacaktır. create_earth() işlevi, modül arayüzü üzerinden erişime açık olmayacak ve çağrıldığında bir istisna oluşturacaktır (bu istisnayı yakalayabilir ya da yakalayamayabilirsiniz; bu sadece öğretim amaçlıdır). Yine kasıtlı olarak bir mypy hatası da ortaya çıkacaktır.
- ft_alembic_5.py dosyası, alchemy modülüne erişmek için from alchemy import ... yapısını kullanacak ve ardından air'i oluşturacaktır.



_ /init/_ .py dosyanızda flake8 / mypy hatalarıyla karşılaşabilirsiniz. Bu durumda genel kurallara uyun.

Örnek:

```
$> python3 ft_alembic_0.py
=== Alembic 0 ===
'import ...' yapısını kullanarak elements.py dosyasına erişme
create_fire işlevini test etme: Ateş ögesi oluşturuldu

$> python3 ft_alembic_1.py
=== Alembic 1 ===
'from ... import ...' yapısını kullanarak elements.py dosyasına
erişme. create_water işlevini test etme: Su ögesi oluşturuldu

$> python3 ft_alembic_2.py
=== Alembic 2 ===
'import ...' yapısını kullanarak alchemy/elements.py dosyasına erişme
create_earth'i test etme: Toprak ögesi oluşturuldu

$> python3 ft_alembic_3.py
=== Alembic 3 ===
'from ... import ...' yapısını kullanarak alchemy/elements.py dosyasına
erişme create_air işlevini test etme: Hava elementi oluşturuldu

$> python3 ft_alembic_4.py
=== Alembic 4 ===
'import alchemy' kullanarak alchemy modülüne erişme create_air'i
test etme: Hava elementi oluşturuldu
Şimdi, tüm işlemlere erişilemeyeceğini gösterelim Bu, bir
istisna oluşturacaktır!
Gizli 'create_earth' işlevinin test edilmesi: Traceback (en son çağrı en sonda):
"/tmp/Python/module-06/ft_alembic_4.py" dosyası, 12. satır, <module>
    içinde
        print(f'{alchemy.create_earth()}')
        ~~~~~
AttributeError: 'alchemy' modülünde 'create_earth' özniteliği yok. Şunu mu demek istediniz: 'create_air'?

$> python3 ft_alembic_5.py
=== Alembic 5 ===
'from alchemy import ...' kullanarak alchemy modülüne erişme
create_air'i test etme: Hava elementi oluşturuldu
```

IV.2 Bölüm II: Damıtma

İçerik geçmişi import sanatını ustalaştırın ve uzak dosyalardan uzak kodları çağırın.

Laboratuvar mimarimizi geliştirelim ve yeni bir bileşen ekleyelim:

alchemy/potions.py dosyası şunları içerecek:

- “[oluşturulan toprak elementi]” ve “[oluşturulan hava elementi]” ile hazırlanmış şifa iksiri” dizesini döndüren `healing_potion()` işlevi.
- “[oluşturulan ateş elementi]” ve “[oluşturulan su elementi]” ile hazırlanmış güç iksiri” dizesini döndüren `strength_potion()` işlevi.
- Dört temel elemente erişmek için yararlı olabilecek diğer her şey.

Bu yeni işlevleri test etmek için aşağıdaki iki komut dosyasını oluşturun:

- `ft_distillation_0.py` dosyası, `alchemy/potions.py` dosyasına doğrudan erişmek için “`from ... import ...`” yapısını kullanacak ve ardından hem şifa iksirini hem de güç iksirini hazırlayacaktır.
- `ft_distillation_1.py` dosyası, `alchemy` modülüne erişmek için `import alchemy` yapısını kullanacak, ardından orijinal güç iksirini ve şifa iksirinin paket *takma adı* olan özel `heal()` iksirini hazırlayacaktır. `_init_.py` dosyanızı iyileştirmeniz gerekecektir dosyasını iyileştirmeniz gerekecektir.

Örnek:

```
$> python3 ft_distillation_0.py
=== Damıtma 0 ===
alchemy/potions.py dosyasına doğrudan erişim
strength_potion'ı test etme: 'Ateş elementi yaratıldı' ve 'Su elementi yaratıldı' ile hazırlanmış güç iksiri
healing_potion'ı test etme: 'Toprak elementi yaratıldı' ve 'Hava elementi yaratıldı' ile hazırlanmış şifa iksiri

$> python3 ft_distillation_1.py
=== Damıtma 1 ===
İksirlere erişmek için 'import alchemy' yapısını kullanma
strength_potion'ı test etme: 'Ateş elementi oluşturuldu' ve 'Su elementi oluşturuldu' ile hazırlanan güç iksiri heal
alias'ı test etme: 'Toprak elementi oluşturuldu' ve 'Hava elementi oluşturuldu' ile hazırlanan şifa iksiri
```

IV.3 Bölüm III: Büyük Dönüşüm

Mutlak ve göreceli ie aktarmalar arasındaki eski tartışmayı anlayın: aynı sihirli formüle ulaşmanın iki farklı yolu.

Laboratuvar mimarimizi bir kez daha iyileştirelim ve yeni bir bileşen ekleyelim:

alchemy/transmutation/recipes.py dosyası şunları içerecek:

- "Kurşunu Altına Dönüştüren Tarif: '[created air]' ve '[created strength potion]' karışımını '[created fire]' ile demleyin" dizesini döndüren `lead_to_gold()` işlevi.
- En az bir mutlak ie aktarma ve bir göreceli ie aktarma.

Her şeyin başarıyla ie aktarıldığını doğrulamak için üç test betiği oluşturun:

- `ft_transmutation_0.py` dosyası, `import ...` yapısını kullanarak `alchemy/transmutation/recipes.py` dosyasına doğrudan erişecek ve ardından kurşunun altına dönüştürülmesini sağlayacaktır.
- `ft_transmutation_1.py`, `import ...` yapısını kullanarak `transmutation` modülünü doğrudan ie aktaracak ve ardından kurşunun altına dönüştürülmesini sağlayacaktır.
- `ft_transmutation_2.py`, `import ...` yapısını kullanarak `alchemy` modülünü doğrudan ie aktaracak ve ardından kurşunun altına dönüştürülmesini sağlayacaktır.

Bu komut dosyalarını tamamlamak için birkaç ek dosya oluşturmanız ve/veya güncellemeniz gerekebilir.

Örnek:

```
$> python3 ft_transmutation_0.py
=== Transmutasyon 0 ===
alchemy/transmutation/recipes.py dosyasını doğrudan kullanma
Kurşunu altına dönüştürme testi: Kurşunu altına dönüştüren tarif: 'Hava elementi oluşturuldu' ve 'Güç iksiri
    hazırlandı' ile 'Ateş elementi oluşturuldu' ve 'Su elementi oluşturuldu' karışımından 'Ateş elementi oluşturuldu'
    ile hazırlanan iksir

$> python3 ft_transmutation_1.py
=== Dönüşüm 1 ===
Transmutasyon modülünü doğrudan içe aktar
Kurşunu altına dönüştürme testi: Kurşunu altına dönüştürme tarifi: "Hava elementi oluşturuldu" ve "Güç iksiri
    hazırlandı" ile "Ateş elementi oluşturuldu" ve "Su elementi oluşturuldu" karışımını "Ateş elementi
    oluşturuldu" ile karıştırın

$> python3 ft_transmutation_2.py
=== Dönüşüm 2 === Yalnızca
simya modülünü içe aktarın
Kurşunu altına dönüştürme testi: Kurşunu Altına Dönüştürme Tarifi: "Hava elementi yaratıldı" ile "Ateş elementi
    yaratıldı" ve "Su elementi yaratıldı" ile hazırlanan "Güç iksiri"ni "Ateş elementi yaratıldı" ile karıştırın
```



Bir simyacı ne zaman mutlak yolları, ne zaman göreceli yolları kullanmalıdır?

IV.4 Bölüm IV: Patlamayı Önlemek

Modüllerin birbirlerini sonsuz bir döngü içinde çağırmaya çalışması sonucu ortaya çıkan ve laboratuvarınızı yok etme tehdidi oluşturan patlayıcı döngüsel bağımlılık lanetini tespit etmeyi ve kırmayı öğrenin!

Son olarak, laboratuvar mimarimize şu son bileşenleri ekleyeceğiz:

alchemy/grimoire/light_spellbook.py dosyası şunları içerecektir:

- Işık büyüsü için izin verilen bileşenlerin bir listesini döndüren `light_spell_allowed_ingredients()` işlevi; örneğin “toprak”, “hava”, “ateş”, “su”.
- `light_spell_record(spell_name: str, ingredients: str)` işlevi, büyüün kaydedilip kaydedilmediğini veya reddedilip reddedilmediğini belirten bir dize döndürür. Karar, aşağıda açıklanan doğrulama işlevinden gelir.

alchemy/grimoire/light_validator.py dosyası şunları içerecektir:

- Bileşenleri ve “VALID” veya “INVALID” anahtar sözcüğünü içeren bir dize döndüren `validate_ingredients(ingredients: str)` işlevi. Bileşenler, büyü kitabında izin verilen bileşenlerden en az birini içeriyorsa (büyük/küçük harf duyarlı değildir) geçerlidir.

Şimdi her iki dosyayı da kopyalayarak `dark_spellbook.py` ve `dark_validator.py` dosyalarını oluşturun ve fonksiyon adlarını da değiştirin. Diyelim ki kara büyü şu bileşenleri kullanıyor: “yarasalar”, “kurbağalar”, “arsenik” ve “göz küresi”.

Şimdi birkaç büyüyü test etme zamanı! Işık büyüsünün döngüsel bağımlılıklardan ve laboratuvar patlamalarından kaçındığını, karanlık büyüün ise tehlikeli olduğunu ve döngüsel bağımlılıklar nedeniyle patladığını göstermek için iki komut dosyası oluşturun:

- `ft_kaboom_0.py`, `grimoire` modülüne doğrudan erişecek ve ardından bir ışık büyüsünü hatasız bir şekilde kaydedecektir. Döngüsel bağımlılıklardan kaçınmanın birçok yolu vardır; bunlardan birini seçin. Değerlendirme sırasında farklı yaklaşımları açıklamaya hazır olun.
- `ft_kaboom_1.py` dosyası, gizlice `dark_spellbook.py` dosyasına doğrudan erişecek ve ardından bir karanlık büyü kaydetmeye çalışacaktır. Bu işlem başarısız olmalı ve bir istisna oluşturmalıdır (bunu yakalamayı tercih edebilirsiniz ya da etmeyebilirsiniz; bu sadece eğitim amaçlıdır); bu, simyacı laboratuvarınızın az önce patladığını gösterir.

Örnek:

```
$> python3 ft_kaboom_0.py
=== Kaboom 0 ===
Grimoire modülünü doğrudan kullanma
Işık büyüsünü kaydetme testi: Büyü kaydedildi: Fantazi (Toprak, rüzgar ve ateş - GEÇERLİ)

$> python3 ft_kaboom_1.py
=== Kaboom 1 ===
Alchemy/grimoire/dark_spellbook.py dosyasına doğrudan erişim
Şimdi içe aktarmayı test edin - BU, YAKALANAMAYAN BİR İSTİSNAYA
NEDEN OLACAKTIR
İzleme (en son çağrı en son):
Dosya "/tmp/Python/module-06/ft_kaboom_1.py", satır 7, içinde <module>
  from alchemy.grimoire.dark_spellbook import dark_spell_record
"/tmp/Python/module-06/alchemy/grimoire/dark_spellbook.py" dosyası, 2. satır, <module> içinde
  from .dark_validator import validate_ingredients
"/tmp/Python/module-06/alchemy/grimoire/dark_validator.py" dosyası, 2. satır, <module> içinde,
  .dark_spellbook modülünden dark_spell_allowed_ingredients içe aktarılır
ImportError: kısmen başlatılmış 'alchemy.grimoire.dark_spellbook' modülünden
'dark_spell_allowed_ingredients' adı içe aktarılamıyor (büyük olasılıkla dögüsel içe aktarma
nedeniyle) (/tmp/Python/module-06/alchemy/grimoire/dark_spellbook.py)
```

Bölüm V

Teslim Talimatları

Ödevinizi her zamanki gibi Git deponuzda teslim edin. Savunma sırasında yalnızca deponuzdaki çalışmalar değerlendirilecektir.

Beklenen dosya ağacını Git deponuzun kök dizinine yerleştirin.



Değerlendirme sırasında, içe aktarma mekanizmalarını açıklamamız, farklı içe aktarma stillerini göstermeniz veya simya laboratuvarınızı değiştirmeniz istenebilir. Yalnızca bunların uygulanışını değil, dört kutsal gizemi de anladığınızdan emin olun.